Aggregation
Composition
Association

⇒ ① How do they differ from each other?
② How to implement them in practice?

Composition → Refers to an object whose existence is completely dependent on the outer object.

```
class Building {
        Room rooms[10]; // Composition
                        // "Has-A" relationship
    :
};
```

```
class Room {
        Seat seats[100]; // Seats have some independent existence even if a room is not present. So this is considered "aggregation".
    :
};
```

Are there any different ways of implementing them?

In case of composition, it is acceptable to initialize the objects in the outer class. But in case of aggregation without composition, this should not be done.

```
class Room {
        Seats *seats = NULL;
        Room() { Seat seat = new Seat;
                 seats = seat; }
};
```

Many types of associations are possible that cannot be represented as aggregation or composition. These cases are also implemented using pointers, since independent existence has to be enforced.

```
class Road {
    :
    Campus *C;
    :
};
```

```
class Campus {
    Road *r;
    :
};
```

Association ⇒ Implementation in C++ is via pointers.

Specialization ⇒ A sub-type of a specific class (Inheritance is the standard way)

Exception Handling in C++

The goal is to provide a standard way by which an error or unexpected situation could be handled.

```
try {
    :
} catch (exception e) {
    cout << e;
} finally { ... }
```

What is this??

Programs where the catch block is invoked run much slower. So it should not be used as a natural part of the logic.

class arrayOutOfBoundException
⋮
⇒ Define exceptions by inheriting the "exception" class.

```
try {
    throw new arrayOutOfBoundException(...);
} catch (arrayOutOfBoundException e) {
}catch (exception e) {...
}catch (divideByZeroException e) {
    ⋮
}
```

Three keywords: ⇒ "try", "throw", "catch"

Each exception has to inherit from the exception class.

The generic requirement of handling any object is managed by (void *) pointer. But special classes like exceptions need to be inherited.

Any other problems if we use exceptions??

① ⇒ Nested exceptions work; but make the code harder to understand.

② Avoid situations where you need to continue running the code from the point of exceptions.

The concept of namespace

```
using namespace std;
```

The most common external library in C++ is the boost library.

```
namespace myNS{
    class Room {
        :
        :
    };
}
using namespace myNS; // or myNS::Room
```

Namespaces provide a logical organization of the content.